CPSC Lecture 12

Uncomputable Functions

- Morning Quiz: (halts? proc input) → return #t if halts.
  #f if infinite loop.

  (proc inputs)
  ↓

P1: Easy

P2:
```
(define (p proc)
  (if (halts? proc proc)
      (factorial 3)
      (factorial -1)))
```
→ #f

```
(define (factorial n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))
```
⇒ #t

(halts? procedure? procedure?) ⇒ does (procedure? procedure?) halt?

↳ Q: ⇒ procedure? a procedure?
A YES/NO
↳ H: It halts.

↓
#t

(halts? number? number?) ⇒ #t

(halts? p procedure?) ⇒ (halts? procedure? procedure?)
↳ #t
↳ gives an answer
↳ halts = #t
↑.

(halts? p number?) ⇒ #t same idea as above.

(halts? p p) ⇒ ~~#~~ (halts? p p)
(p p) ⇒
↳ (p p) ⇒ (halts? p p)
↳ ....

↳ enters this infinite loop.
↝ answer ⇒ undetermined.

I need a bit more context on the halting problem.

The Halting Problem

↳ David Hilbert's challenge: Entscheidungsproblem: algorithm that takes in any logical statement & tell you if it is true or false.

↳ what is logic?

↳ linked to Godel, Church's computability; now it seems to me that computer science is an extended meditation on math

↳ Alan Turing

↳ what is a machine?

↳ Turing Machine.

↳ Computable problems are computable by Turing Machines

↳ There are certain problems that Turing machines cannot solve.

↳ Such as the Halting Problem.

Hence not all logical statements have computable truth values.

Hence

↳ Addressed Hilbert's Challenge.

OK, I predict that these problems are deep fundamental problems that will take days if not weeks for me to fully understand, for the time being I'll just construct the outline

the essence of computer science in computation (math)

CS is Algorithmic math.

The idea of the problem in the morning quiz is:

halts? p p.

p halts on p

Suppose ∃ a machine that tells you whether a procedure halts on an input, then for any procedure & any input, this machine should ~~dotes~~ output #t or #f.

However, ∃ a procedure & an input for which the machine cannot decide in a finite # of steps whether it is #t or #f.

The logic statement here is p halts on input.

Hence, this machine does not exist.

∃ a logical statement for which no machine can determine its truth value! ⇒ Solved the Entscheidungsproblem

Note on Turing Machines & Computability.

- Turing Machines can be encoded in binary.

For example 001 01 010 11 0

001: current state
01: current symbol
010: next state.
11: next symbol
0: dir

- Computability:

A function $f: \{0,1\}^* \to \{0,1\}^*$ is computable if ∃ a Turing machine that, when run with x on its input tape, halts with f(x) on its output tape, for all inputs x.

- Countable sets:

Countable sets: if you can make an infinite ordered list that contains all elements of a set, and that set is no larger than the set of natural numbers. ∈ {1, 2, 3 ... ∞}
↳ ish that a bit arbitrary

↳ Diagonalization: a technique to help you prove there are more real # than natural.

1. 0.①5278943
2. 0.7②8496632
3. 0.57③8581 76
4. 0.694③54873
5. 0.9162④3798
6. 0.4454667189.
⋮

1
2
3
4
5
5
7
⋮

For any scheme you give, I can find a # not on the list (Proof by contradiction)

∴. Hence the real numbers between 0 & 1 are not countable

CPSC lecture 13

① which of the equations hold?

$\overline{x+y} = \bar{x} \cdot \bar{y}$

- ☒ $1-(x+y) = (1-x)+y$ — NOT$(x$ OR $y) =$ NOT$(1-x)\cdot(1-y)$
- ☑ $x*x = x$
- ☑ $x*(y+z) = x*y + x*z$
- ↘ ☑ $1-(x*y) = (1-x)+(1-y)$ — $\overline{x \cdot y} = \bar{x} + \bar{y}$

mistaken this one ↘

- ☑ $1-(x+y) = (1-x)*(1-y)$
- ☑ $x*(1-x) = 0$
- ☑ $1-(1-x) = x$

② Suppose that

$r = p*(1-q)$ ⇒ NOT both sides $\bar{r} = \bar{p} + q = q + (1-p) = 1$

and

$q + (1-p) = 1$

Hence $r = 0$

Then we must have $r =$ $\boxed{0}$? (0 or 1)

Theory

↳ The Halting Problem: 1st example of an uncomputable function.

↳ & there is no general procedure to tell if a Turing Machine halts for all inputs.

↳ there might be inputs for which the Turing Machine × halts, but we have no formal proof for it!

* The Collatz Conjecture: For all integers n, (collatz n) halts, returning 1

```
(define (collatz n)
  (cond
    [(= n 1) 1]
    [(even? n) (collatz (/ n 2))]
    [(odd? n) (collatz (+1 (* n 3)))]))
```

tested up to 2^{71}.

how did ppl come up w/ this?

↳ Generalizing ~~uncomputable~~ uncountability to functions

The function $f: \{0,1\}^* \to \{0,1\}^*$ one not countable.

1. ⓪001 .... $f(1)=1$
2. 0①11 .... $f(2)=0$
3. . . ⓪ . . .

} → We can again use diagonalization to obtain an f that is not on the list

↳ However, Turing machines are countable ← I didn't understand why you can't use the diagonalization trick here & say: aha! ∃ a new Turing machine that is not on the current list & claim that Turing machines are uncountable

↳ Conclusion: because there are more problems than possible programs, most problems are uncomputable.

Boolean Expressions

-Boole's Question = Can we reduce deduction to calculation?

↳ Key idea: let variables stand for propositions, not just numbers.

↳ An Investigation of the Laws of Thought.

CPSC Lecture 14

Morning Quiz:

1. Write A interms of D, B, K.

D: car door closed.

B: driver seat belt fastened.

K: key in ighition

A: safety alarm beeping

Alarm goes off when. key in ighition before you close the doors
or
drive off without a seatbelt

$$A = \bar{D}K + \cancel{D}K\bar{B} = K(\bar{D}+\bar{B})$$

2. $P \Rightarrow Q \Leftrightarrow (\text{NOT } P) \text{ OR } Q$

P true → Q true
→ if P true, Q must be true
→ if P false, I don't care about Q

For which values of x & y will you have ⇓

$(x \Rightarrow y) \text{ AND } (y \Rightarrow x) = 1$? Hint: find the "promise analogy" to be quite useful.

$(\bar{x}+y)\cdot(\bar{y}+x) = 1$

both have to be 1.

x	y	$\bar{x}+y$	$\bar{y}+x$
0	0	1	1
0	1	1	0
1	0	0	1
1	1	1	1

→ so $x=y=1$ or $x=y=0$

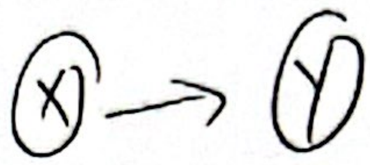
3. Boolean expression in sum of products form.

* For any possible truth table, it is possible to write a Boolean Expression to represent that truth table.

CPSC lecture 15

Not much info.

CPSC lecture 16



Morning Quiz:

Q: why is there a connection btwn X & Y

1.) 2) Easy

3) Design neurons such that $Y = NOT\ X$.

$Z = aX + bNOT\ X$

} synapse from X→Y : strength $\underline{-1}$?

z fires if incoming signal exceeds $\underline{-0.5}$?

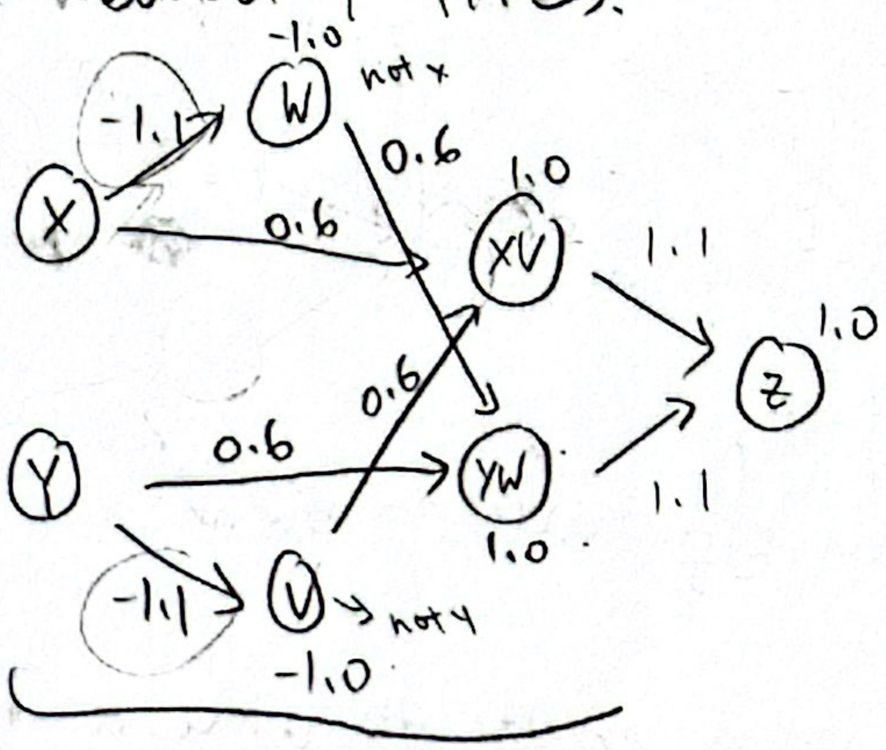
- Neural Networks

↳ because of the "all-or-none" character of nervous activity, neural events & the relations among them can be treated by means of propositional logic.

CPSC lecture 17

Morning Quiz.

Design a neural network such that neuron z fires when neuron X XOR neuron Y fires.



Let this sink in

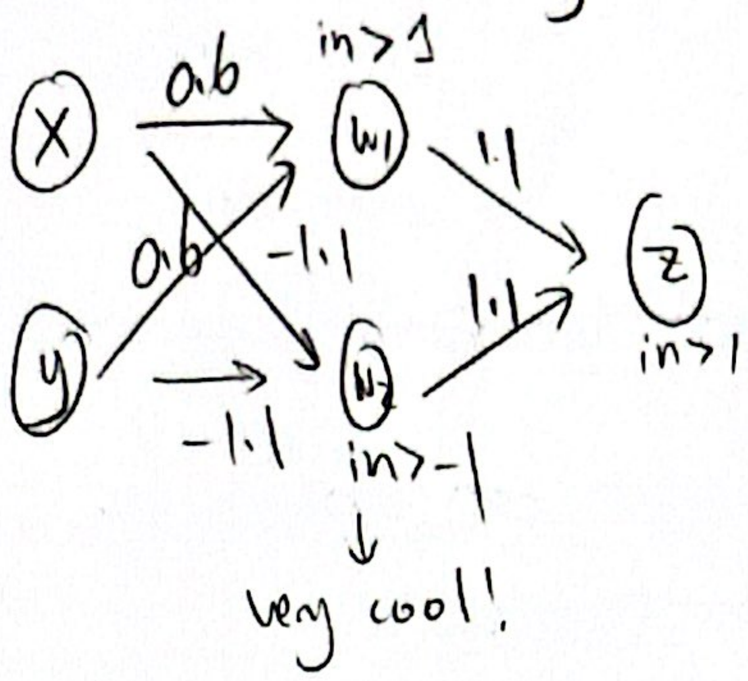
```
(define xor-net
  (net '(x y) '(z)
    (list
      (neuron 'w '(x) '(-1.1) -1.0)
      (neuron 'v '(y) '(-1.1) -1.0)
      (neuron 'xv '(x v) '(0.6 0.6) 1.0)
      (neuron 'yw '(y w) '(0.6 0.6) 1.0)
      (neuron 'z '(xv yw) '(1.1 1.1) 1.0))
```

Neural Networks Continued

* Any Boolean function can be encoded as a neural network.

↳ For example, equality checking ($\overline{XOR}$).

& any (binary) neural network computes some Boolean function.

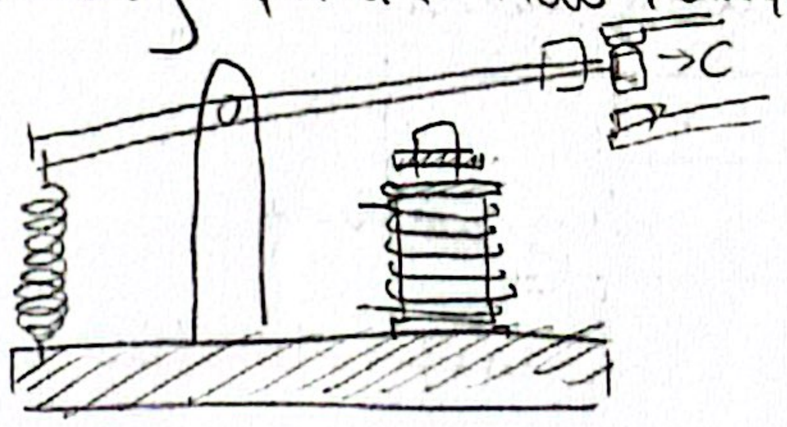


Neural Networks $\Leftrightarrow$ Boolean Logic $\Leftrightarrow$ Electrical Circuits

Claude Shannon

"A Symbolic Analysis of Relay and Switching Circuits"

*Random Interesting Point: How relays work.



Lecture 18

- Morning Quiz: learning perceptrons.

- Frank Rosenblatt: perceptron as a model for how brain might store & process information

- Multi-Layer Perceptron (MLP)
  - ↳ Hidden layers: let the network learn intermediate features
  - ↳ An MLP with one hidden layer can approximate any function (Universal Approximation Theorem)
  - ↳ Deep learning = many layers of simple perceptron-like units

- Training deep nets: gradient descent
  - ↳ define a loss function = measuring how wrong the network is, then nudge all weights to shrink it.

finite-differences gradient descent

For each weight w.

1. increase w by a tiny amount ε
2. measure: $\Delta loss = loss(w+\epsilon) - loss(w)$
3. if $\Delta loss < 0$: increase w; else decrease w;

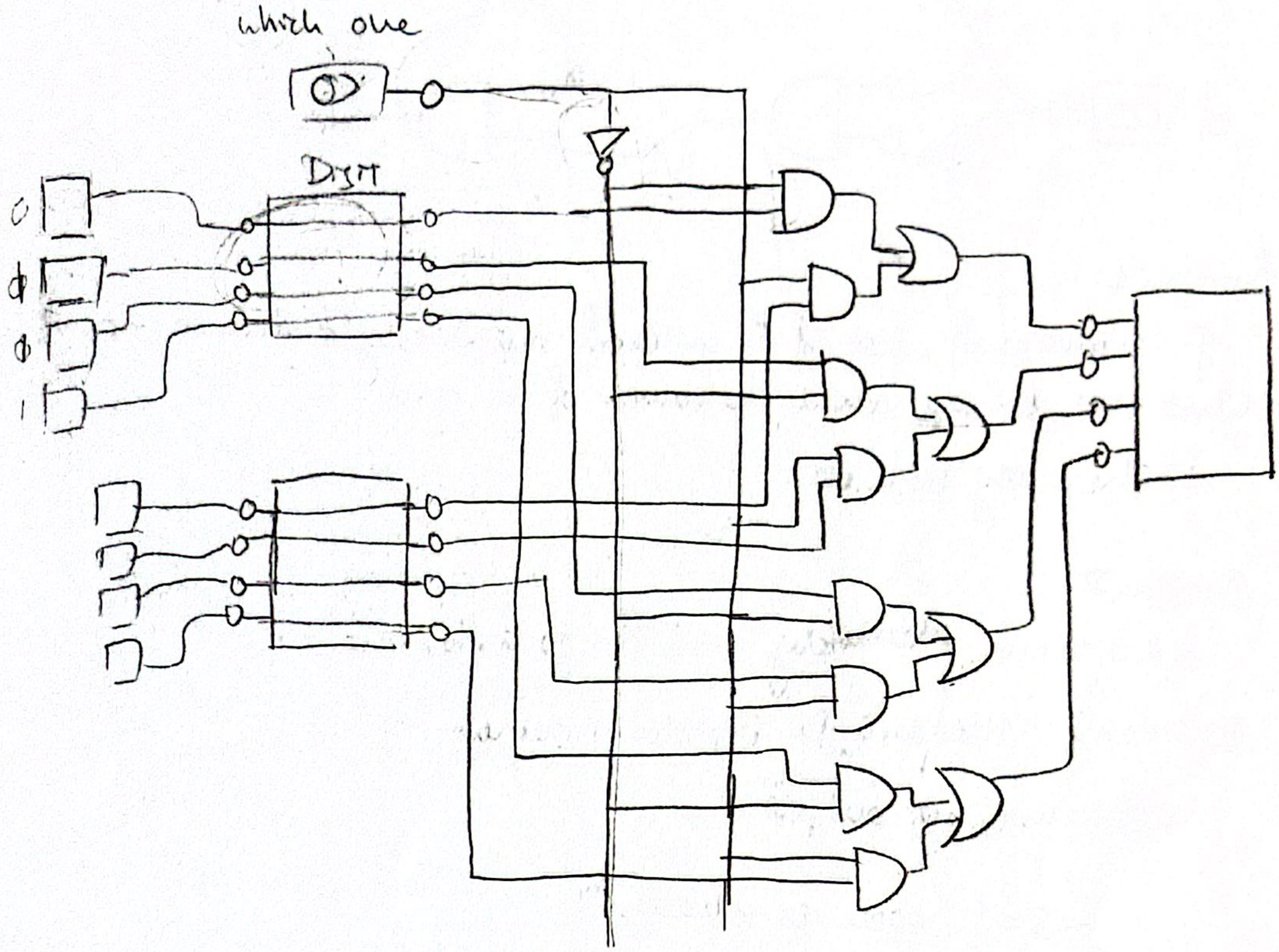
(update: $w \leftarrow w - \alpha(\Delta loss/\epsilon)$.

↳ backpropagation: computes all gradients at once using the chain rule → made deep learning practical

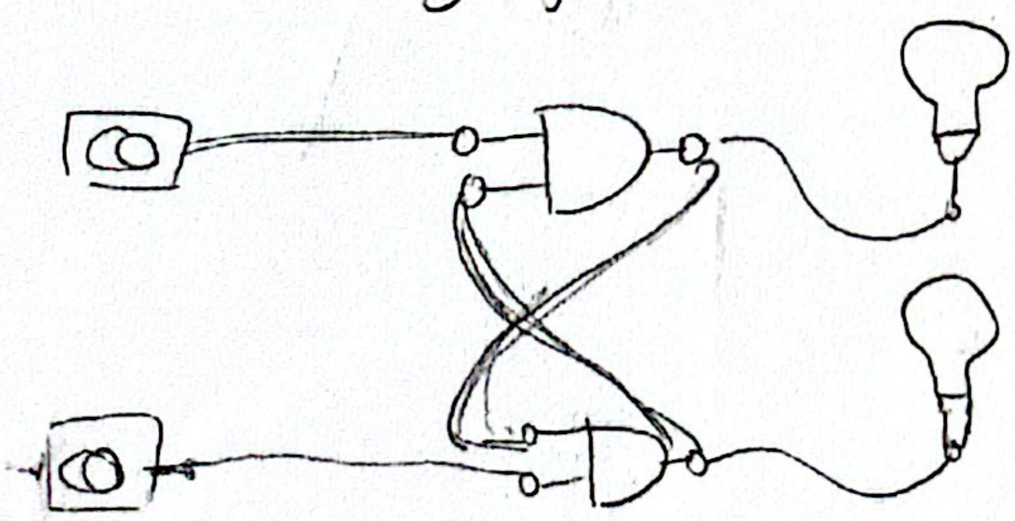
CPSC lecture 19

Morning Quiz Easy

- Gate delays
- Selector Circuit:

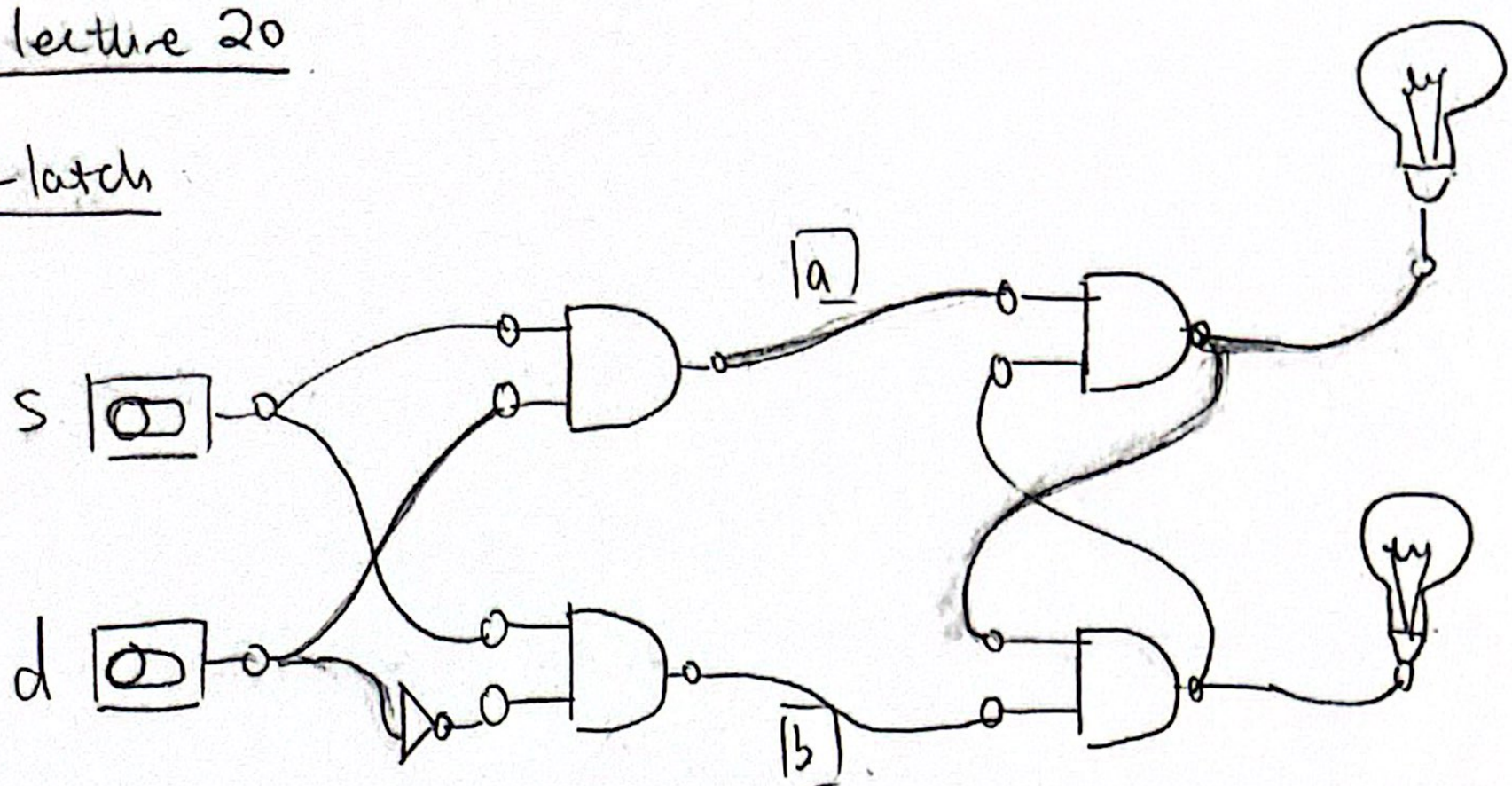


- NAND latch: a key ingredient in better memory



CPSC lecture 20

D-latch



Question 1:

If s remains off, but d is switched repeatedly from on to off, what can you say about the values of a and b?

↳ They stay both on.

Question 2:

d & s turned alternately on & off ⇒ a, b, out --

Question 3: How does D Flip-Flop implement 1-bit memory?

↳ Allows us to set output

↗ S=1 open to change

↘ S=0 circuit locked

store value

↗ d=1

↘ d=0.

PSC lecture 21

Morning Quiz. Interpreting ~~assembly~~ assembly language

↳ no particular knowledge needed.

Lecture 2541

Using skipzero/skippos on if statements

Storei : store ACC to the i^{th} line

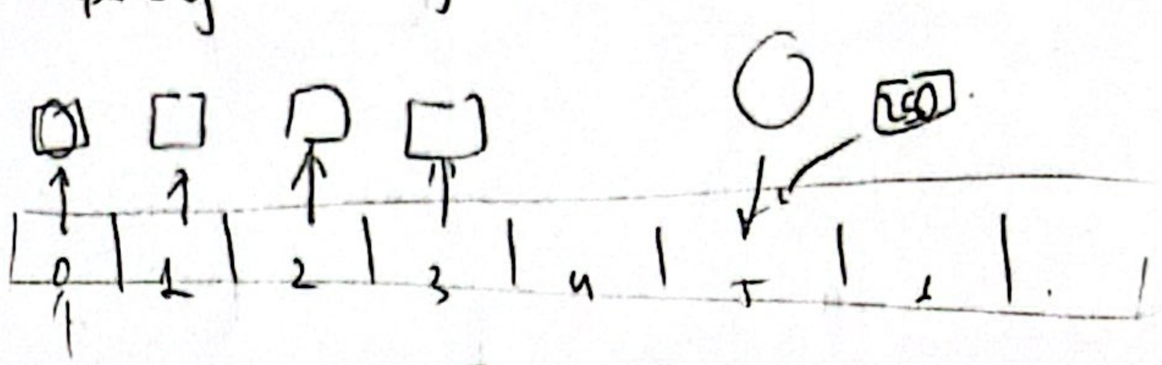
what is this line i? It's the value ~~of~~ stored in the address you are pointing to.

Store address

drawer

like mysheelf analogy

usually current is read as an address



ACC => piece of paper

Store 5

Storei 5

drawer -

which drawer should I open?

& put the piece of paper in?

store: open drawer 5

take out the piece of paper in 5

i:i & store my ACC in the drawer

TC-201 Integer Representation

How to add 2 numbers in this representation?

→ if they have the same sign ⇒ add magnitudes

→ if different signs:

{ determine which # has larger magnitude
subtract smaller mag. from larger mag.
use sign of larger #

It's a very interesting way to think about 2's complement

Other Integer

0 1 2 3 4 5 6 7
-4 -3 -2 -1

Two's Complement

Use "top half" to represent negative numbers?

→ Adding 1 ⇒ still moving right within range [-4,3]

→ Sub 1 ⇒ still moving left within range [-4,3]

lecture 24

→ Using pointers to store a list of data in an array with TC-201

```
loop:  input
       storei pointer     } store value @ pointer addr
       load pointer
       add one
       store pointer      } update pointer addr
       jump input
```

Lecture 25

→ One way to expedite programming in assembly is to first write the program in a higher-level language then translate it to assembly